UNIT 4

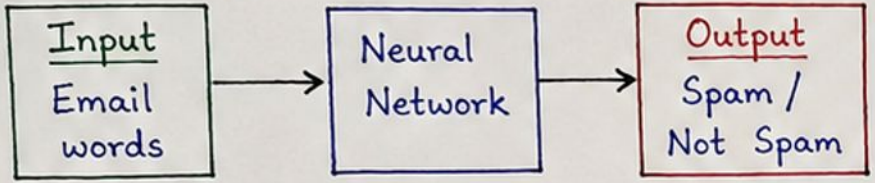
Recurrent Neural Networks

What is a Neural Network?

A Neural Network (NN) is a machine learning model inspired by the way the human brain processes information.
It consists of connected nodes (neurons) that learn patterns from data and use those patterns to make predictions or decisions.

Simple Example :

Suppose we want to predict whether an email is Spam or Not Spam.

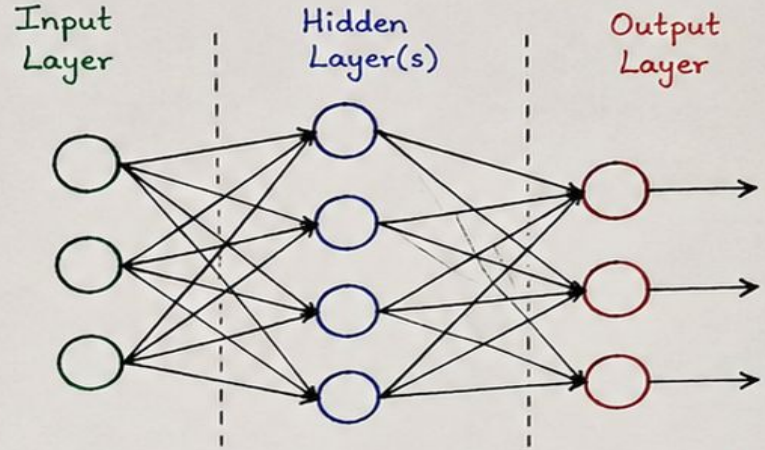


The network learns which words and patterns are commonly associated with spam.

Basic Structure

A neural network usually has 3 types of layer

Input Layer → Hidden Layer(s) → Output L



1. Input Layer

Receives the data.

Example :

- Word features
- Age
- Salary
- Image pixels

2. Hidden Layer(s)

Process the input and learn useful patterns / features.
A network can have one or many hidden layers.

3. Output Laye

Produces the fin result.

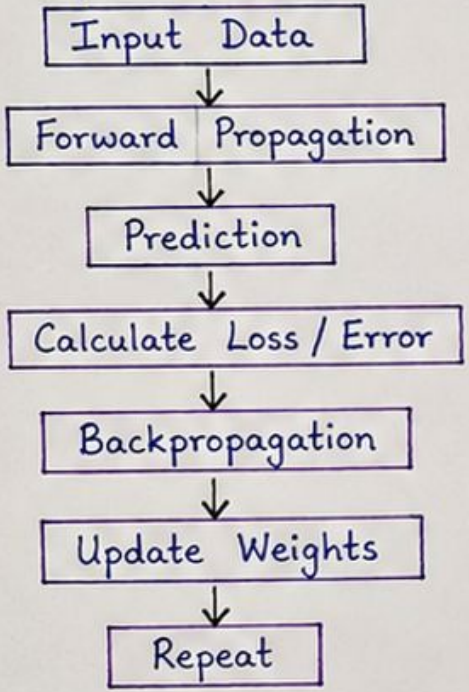
Example :

- Positive / Negat Neutral
- Spam / Not Sp
- Cat / Dog

How Does a Neural Network Learn?

Each connection has a weight.

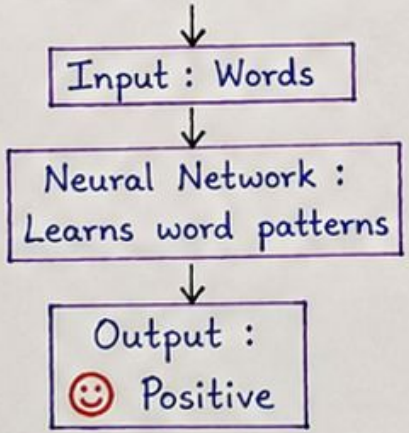
During training:



The goal is to reduce the error (loss) and improve predictions.

Very Simple Example :

"This movie is excellent."



Key Terms

* Neuron : Small processing unit
* Weight : Importance given to an input
* Bias : Helps adjust the output
* Activation Function : Decides how strongly a neuron activates
* Loss : Measures prediction error
* Gradient : Shows how weights should be changed
* Backpropagation : Sends error backward to update weights

What Is NLG?

Natural Language Generation (NLG) is a part of Natural Language Processing (NLP) that enables computers to automatically generate human-readable text.

Connection between NLG and Language Model

NLG can be implemented using a Language Model (LM).

A language model gives NLG systems a way to decide, statistically, which words should come next.

What Is a Language Model?

A Language Model learns the probability of word sequences and predicts which words are likely to come next.

"The cat is small." — More probable and grammatically correct.

"Small the is cat." — Less probable and incorrect word order.

A language model learns that some word sequences are far more likely to occur than others.

The Basic Idea

How a language model turns context into a prediction



EXAMPLE

"The cat is ___" → **"small"**

Types of Language Models

Statistical Language Models

Use **statistical methods** to calculate the probability of word sequences.

EXAMPLES

- **N-gram models**
- **Hidden Markov Models (HMM)**
- **Linguistic rules**

Neural Language Models

Use **neural networks** to learn language patterns and predict words.

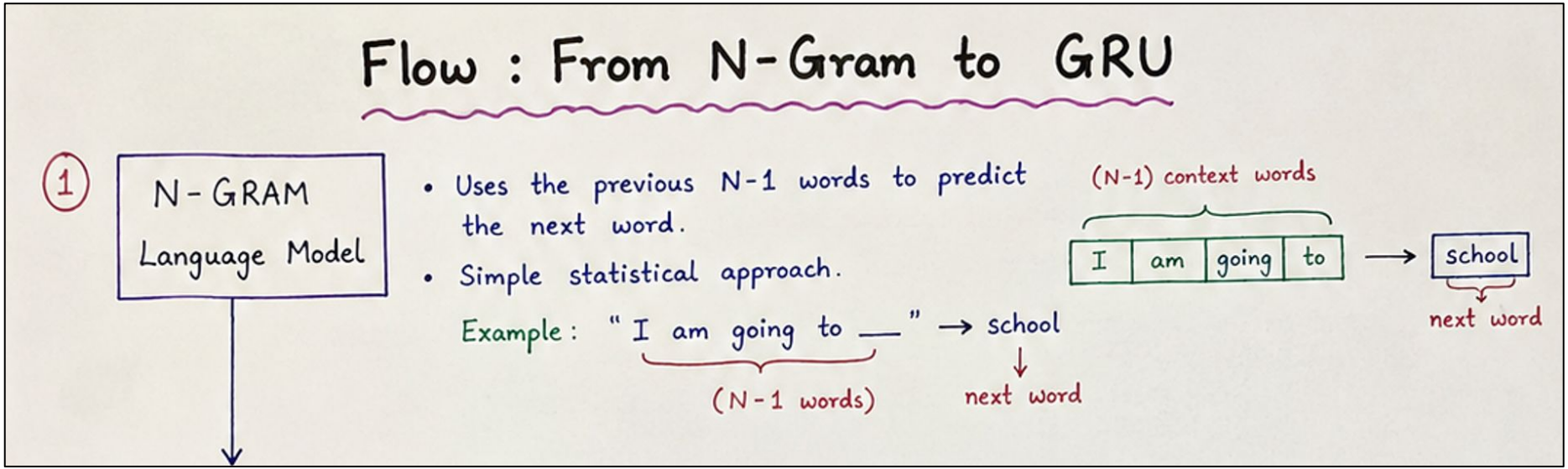
EXAMPLES

- **RNN**
- **LSTM**
- **GRU**
- **Transformer**

The Evolution of Language Models

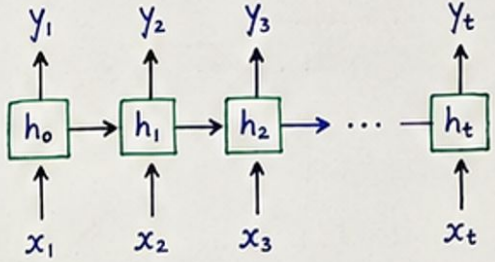
N-Gram → RNN → Vanishing Gradient → LSTM → GRU

The Evolution of Language Models



2 Recurrent Neural Networks (RNN)

- RNN is a type of neural network designed for sequence data.
- It keeps a "hidden state" that stores information from previous words.
- Hidden state is updated at each time step.

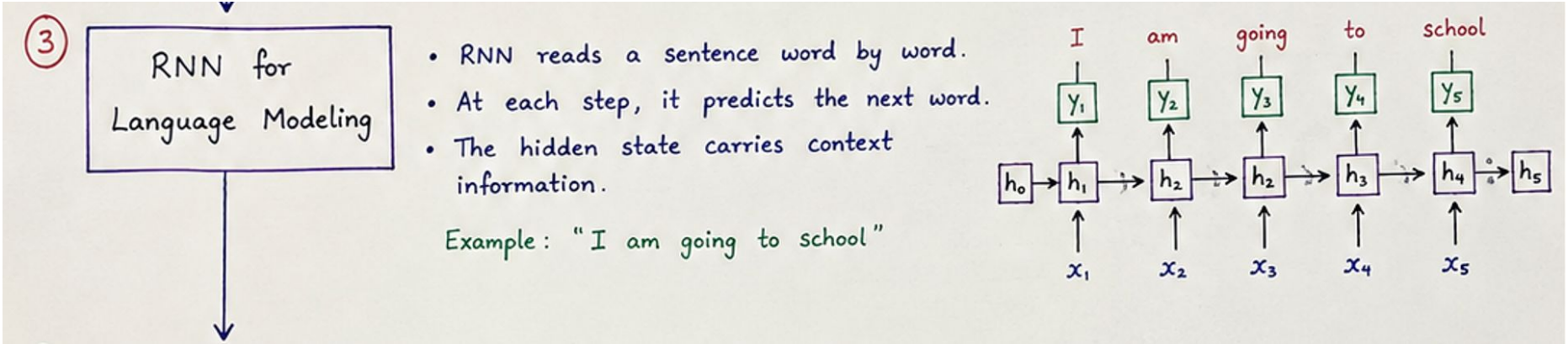


x_t : input at time t (word)
h_t : hidden state at time t
y_t : output at time t (next word)

EXAMPLE — READING WORD BY WORD

I → **am** → **going**

hidden state carries "I am" forward as context



EXAMPLE

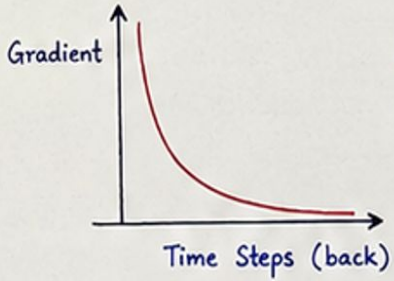


Each word is predicted from all the words before it — "school" follows from "I am going to".

4

Vanishing Gradient Problem

- In basic RNN, gradients shrink as we backpropagate through many time steps.
- Early information (from far back) is forgotten.
- RNN finds it hard to learn long-term dependencies.

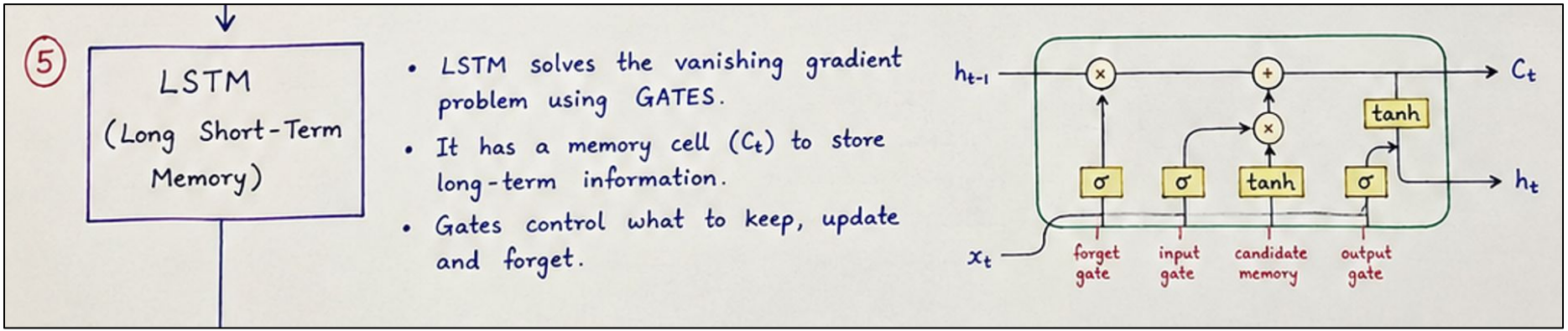


Gradients go to very small values as we go back.

EXAMPLE "The cat, which had been missing for days, finally ___"

cat

"The cat, which had been missing for days, finally ___" — by the time the RNN reaches the blank, "cat" has faded from memory.



Forget Gate	Input Gate	Output Gate
Decides what old information to remove	Decides what new information to store	Decides what to output at this step

EXAMPLE

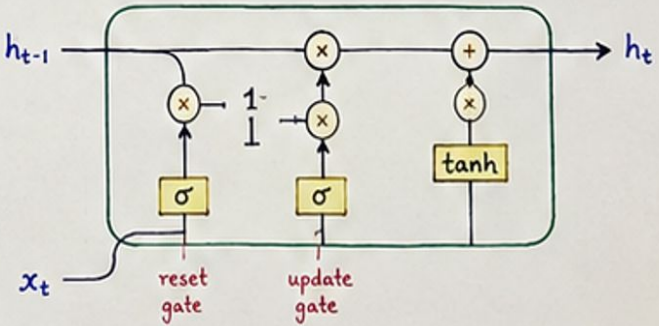
"The cat, which had been missing for days, finally ___"

LSTM remembers **"cat"** from earlier and predicts **"returned"** instead of losing the context.

6

GRU (Gated Recurrent Unit)

- GRU is a simpler variant of LSTM.
- It has fewer gates (update gate and reset gate).
- It is computationally cheaper and often performs similarly to LSTM.



Update Gate

How much of the past to keep vs. replace with new info

Reset Gate

How much of the past to forget when computing new content

Fewer gates, fewer parameters than LSTM → usually faster and cheaper to train.

EXAMPLE

"The weather today is very ___"

Update gate keeps "weather" as the topic, reset gate focuses on "very" → predicts "hot" / "cold"

N-GRAM LANGUAGE MODELS (Revise Once again)

Counting Words to Predict What Comes Next

What Is an N-Gram?

- An n-gram is a **contiguous chunk of ‘n’ words** pulled from a sentence.
- Language models built on n-grams learn **to predict the next word by counting how often these chunks**

Example sentence

“The students opened their _____”

Type	N-grams extracted
Unigrams (n=1)	"the", "students", "opened", "their"
Bigrams (n=2)	"the students", "students opened", "opened their"
Trigrams (n=3)	"the students opened", "students opened their"
4-grams (n=4)	"the students opened their"

Why N-Gram Matter	Means	Example
1. Context & Semantics	N-grams help understand the meaning of words based on the words around them.	**“I went to the bank to deposit money.”** → The words **“deposit money”** help us understand that *bank* means a **financial institution**, not a river bank.
2. Better Language Models	N-grams help a computer understand common combinations of words.	**“Good morning”** is a common word combination. A language model learns that after **“Good”**, **“morning”** is likely to come next.
3. Text Prediction	The computer uses previous words to predict the next word.	**“I am going to the ___”** → The computer may predict **“market”**, **“school”**, or **“office”** based on what it has learned.
4. Information Retrieval	Search engines can use groups of words to find more relevant information.	If you search **“best restaurants in Mumbai”**, the system looks for pages containing related word combinations such as **“best restaurants”** and **“restaurants in Mumbai.”**
5. Feature Extraction	N-grams can act as clues to understand or classify text.	**“This movie is very good”** → The phrase **“very good”** is a useful feature that can help a system classify the review as **Positive** .

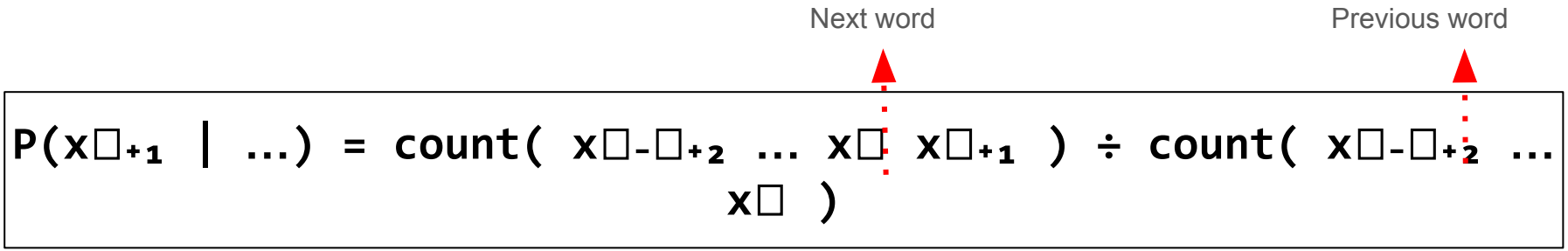
The N-Gram (Markov) Assumption

An n-gram model makes a simplifying assumption: the next word depends only on the previous (n − 1) words — not the entire sentence history.

$$P(x_{t+1} \mid x_t, x_{t-1}, \ldots, x_1) \approx P(x_{t+1} \mid x_t, \ldots, x_{t-n+2})$$

How Do We Compute These Probabilities?

The conditional probability of a word given the previous (n − 1) words is the ratio of two counts:



Probability of the next word =

Number of times the complete word sequence appears ÷ Number of times the previous word sequence appears

The formula says: “How likely is the next word? Count how often the whole sequence occurs and divide it by how often the previous sequence occurs.”

EXAMPLE

Sentence Probability with a Bigram Model

- <s> the cat sat on the mat </s>
- <s> the dog sat on the rug </s>
- <s> the cat ate the fish </s>

$$P(\text{next word}|\text{previous word}) = \frac{\text{Count}(\text{previous word, next word})}{\text{Count}(\text{previous word})}$$

Term	Calculation → Value
P(the | <s>)	3 / 3 = 1.000
P(cat | the)	2 / 6 = 0.333
P(sat | cat)	1 / 2 = 0.500
P(on | sat)	2 / 2 = 1.000
P(the | on)	2 / 2 = 1.000
P(mat | the)	1 / 6 = 0.167
P(</s> | mat)	1 / 1 = 1.000

P(“the cat sat on the mat”) = 1.000×0.333×0.500×1.000×1.000×0.167×1.000 ≈ 0.0278

EXAMPLE

Predicting the Next Word

Using the same toy corpus, the model picks the candidate with the highest conditional probability.

What follows “the”?

Candidate	Count	P(w | the)
cat	2	0.333 ← highest
dog	1	0.167
mat	1	0.167
rug	1	0.167
fish	1	0.167

Prediction
“cat” is the most likely next word after “the” (probability 0.333).

What follows “cat”?

Candidate	Count	P(w | cat)
sat	1	0.500
ate	1	0.500

A tie at 0.5 each — in practice, ties are broken using more context (e.g. moving to a trigram model) or frequency in a larger corpus.

EXAMPLE

A 4-Gram Language Model

“As the proctor started the clock, the students opened their _____”

In a 4-gram model, the missing word w depends only on the previous 3 words — “students opened their” — regardless of anything earlier in the sentence.

```
P(w | students, opened, their) = count(students opened their w) ÷ count(students opened their)
```

Phrase	Count	Resulting probability
“students opened their”	1000	—
“students opened their books”	400	P(books | …) = 0.4
“students opened their exams”	200	P(exams | …) = 0.2

0.4 > 0.2

The 4-gram model predicts **“books”** as the next word.

But Is "Books" Really the Right Choice?

Given the full sentence, the answer feels less obvious:

> **“As the proctor started the clock, the students opened their _____”**

Given the words “proctor” and “clock”, a human reader would likely guess **“exams”** — not “**books.**” But the 4-gram model never saw the word “proctor” at all, b**ecause it only looks 3 words back.**

By fixing n, an n-gram model is forced to discard everything beyond a short, fixed window of context. Here, the crucial clue — “proctor” — **falls just outside that window, so the model settles on the more frequent but less context-appropriate word.**

LIMITATIONS

Why This Happens

Fixed context window

Increasing n to capture more context means far fewer occurrences of any specific n-gram in the corpus — counts become unreliable.

Sparsity problem

Many valid n-grams (e.g., “opened their exams”) may never appear in training, giving them probability zero even if reasonable.

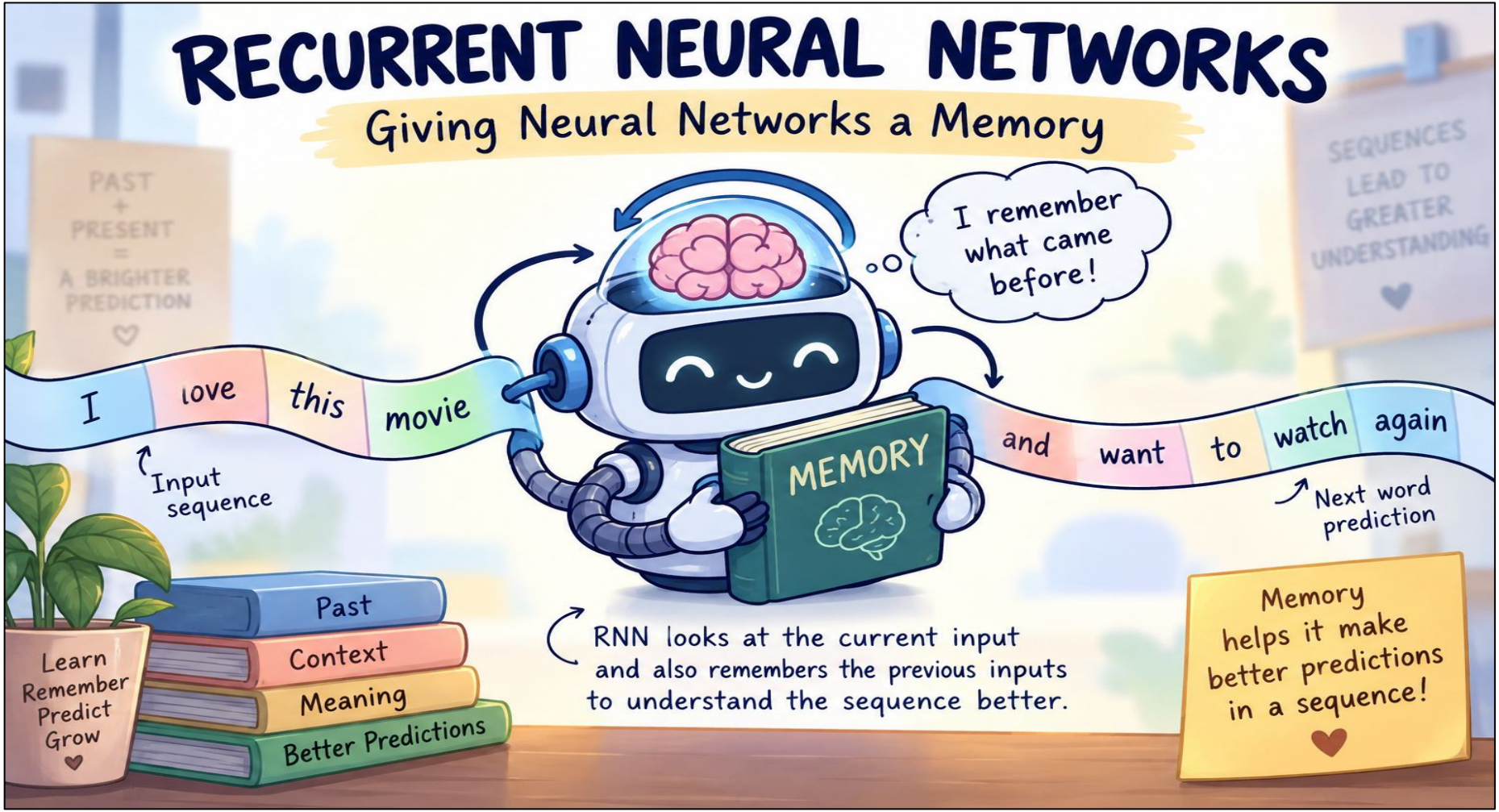
Storage problem

The model must store counts for every n-gram observed in the corpus — memory requirements grow quickly as n increases.

No true understanding of meaning

The model has no notion that “proctor” and “exams” are related — it purely counts surface word sequences.

Recurrent Neural Network

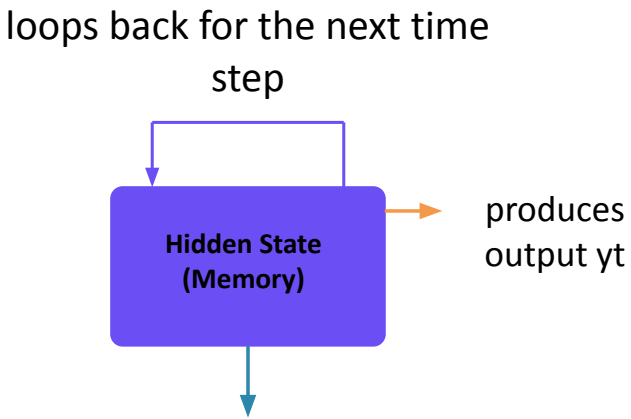

RECURRENT NEURAL NETWORKS
Giving Neural Networks a Memory
PAST + PRESENT = A BRIGHTER PREDICTION
SEQUENCES LEAD TO GREATER UNDERSTANDING
I remember what came before!
I
love
this
movie
Input sequence
MEMORY
and
want
to
watch
again
Next word prediction
Learn
Remember
Predict
Grow
Past
Context
Meaning
Better Predictions
RNN looks at the current input and also remembers the previous inputs to understand the sequence better.
Memory helps it make better predictions in a sequence!

What Is a Recurrent Neural Network?

A Recurrent Neural Network (RNN) is a type of neural network where the output from the previous step is fed back as input to the current step.

Imagine reading a sentence and trying to predict the next word — **you don't rely only on the current word, you remember the words that came before.**

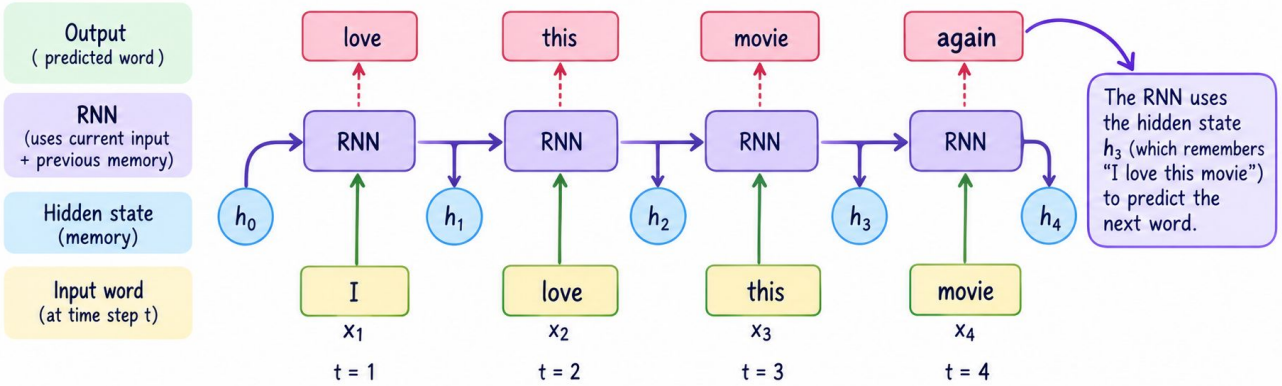
An RNN works the same way: it **"remembers"** past information through **a hidden state**, and considers everything earlier to make its next prediction.



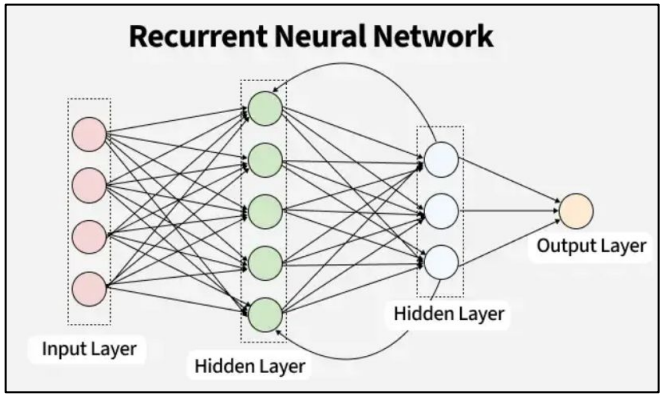
RNNs maintain memory of past inputs, use shared weights across time, and are designed for sequential and temporal data — NLP, forecasting, and speech.

RNN Example: Predicting the Next Word

Input sentence: "I love this movie" → Predict next word: "again"



At each step, the RNN takes the current word and the previous hidden state (memory), updates the hidden state, and produces the next word.



Example: RNN reading a sentence

Suppose the sentence is:

> **"I love this movie"**

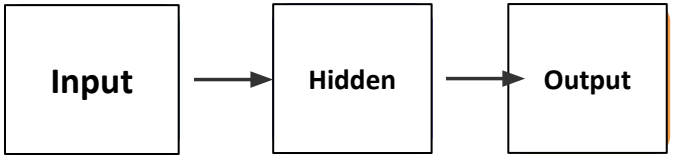
The RNN reads **one word at a time** and keeps the previous information in its **hidden state (memory)**.

Time	Current Input x_t	Hidden State / Memory	Output y_t
$t = 1$	**I**	Remembers "I"	Predicts **love**
$t = 2$	**love**	Remembers "I love"	Predicts **this**
$t = 3$	**this**	Remembers "I love this"	Predicts **movie**
$t = 4$	**movie**	Remembers "I love this movie"	Predicts **again**

Why Not Just Use a Feedforward Network?

In a feedforward network (ANN), **information flows only in one direction** — input, through hidden layers, to output. There are no cycles or loops.

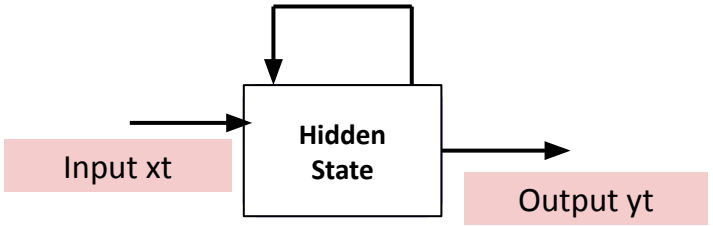
Feedforward Neural Network (FNN)



- **Can't handle sequential data**
- Considers only the current input
- **Can't memorize previous i**nputs

Recurrent Neural Network (RNN)

loop: remembers previous state

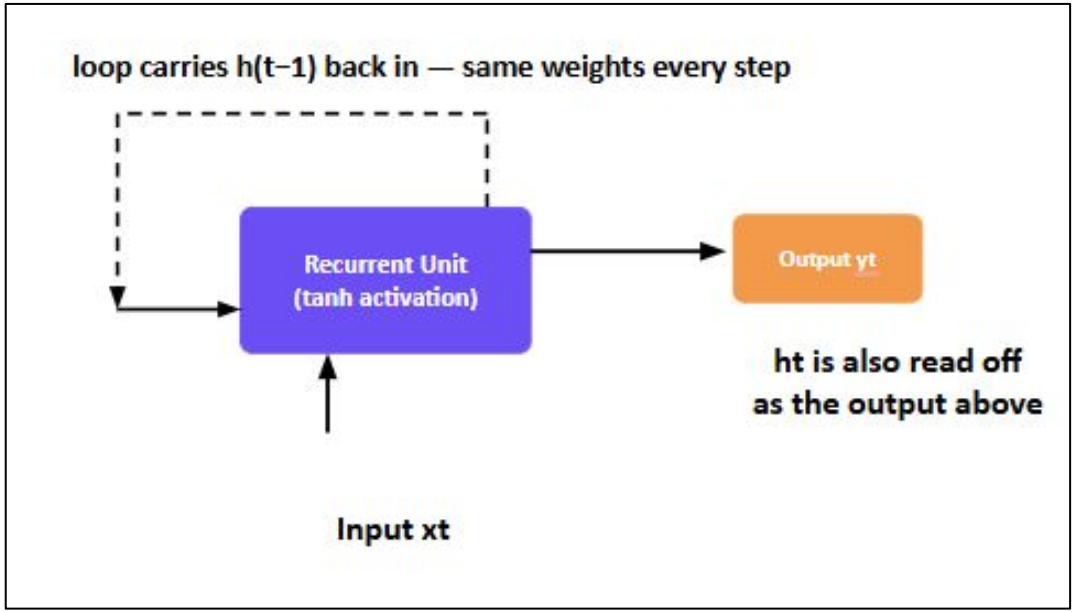


- Feedback loops pass information forward through time
- Maintains memory of past inputs via the hidden state
- Effective for text and time-series data

Sequential = What comes before/after?
Temporal = What happens over time?

Inside the Recurrent Neuron

The fundamental processing unit of an RNN is the recurrent unit. It holds a hidden state that carries information **about previous inputs**, feeding it back so the network can capture dependencies across time.



RNN unfolding = showing the same RNN separately for each time step so that we can train it using BPTT.

Example

For:

> "I love this movie"

The unfolded RNN processes:

```
"I" → RNN → "love"
        ↓
"love" → RNN → "this"
          ↓
"this" → RNN → "movie"
          ↓
"movie" → RNN → "again"
```

Each RNN box represents **one time step**.

What is Backpropagation Through Time?

Once the RNN is unfolded, we can go backward through all the time steps to find and correct errors.

RNN Unfolding – Simple Example

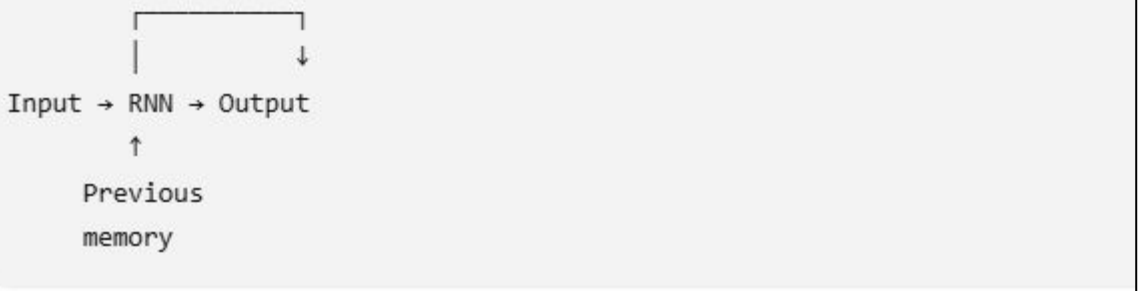
Suppose we want an RNN to understand the sentence:

> **"I love Python"**

An RNN processes the words **one by one**.

1. Normal RNN view

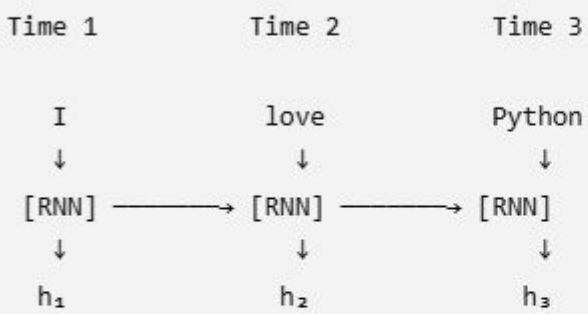
The RNN looks like one loop:



The loop means the **previous information is carried to the next step**.

2. Unfolding the RNN

We open/unfold the loop across time:



Here:

- **t1**: RNN reads **"I"** → produces h_1
- **t2**: RNN reads **"love"** + previous memory h_1 → produces h_2
- **t3**: RNN reads **"Python"** + previous memory h_2 → produces h_3

The **same RNN weights are used at all three time steps**.

Why do we unfold?

Unfolding makes the RNN's work at each time step visible:

```
I → love → Python
↓     ↓      ↓
RNN → RNN → RNN
```

This allows us to see how information moves **forward through time** and how the error can later move **backward through time (BPTT)**.

RNN unfolding means opening the RNN loop and showing the same RNN separately at each time step.

Weight Sharing in RNNs

In an RNN, the **same weights** W_x, W_h, W_y are used again and again at **every time step**.

Think of it like using the **same rule repeatedly**:

```
Word 1 → RNN → Word 2 → RNN → Word 3 → RNN
          ↑               ↑               ↑
     Same weights    Same weights    Same weights
```

Weight	Simple meaning	Connects
Wx	Input weight	Current input `x(t)` → hidden state
Wh	Hidden/recurrent weight	Previous hidden state `h(t-1)` → current hidden state
Wy	Output weight	Hidden state `h(t)` → output `y(t)`

Example

For the sentence:

> **"I love this movie"**

The RNN uses the **same set of weights** to process:

- **I**
- **love**
- **this**
- **movie**

It doesn't create new weights for every word.

Weight sharing = Using the same learned weights at every step.

Why? It allows the RNN to handle short or long sentences without needing a new set of weights for each position.

Because **the same type of work is being done at every time step.**

Simple example

Sentence:

> **I → love → Python**

At each step, the RNN needs to do the same basic job:

Take the current word + previous memory → produce new memory

So we don't need different weights for every word.

```
"I"       → RNN → h₁
              ↑
          Same W

"love"    → RNN → h₂
              ↑
          Same W

"Python"  → RNN → h₃
              ↑
          Same W
```

Why not use different weights?

Suppose we use:

```
"I"       → W₁
"love"    → W₂
"Python"  → W₃
```

Now the network would have to **learn separate rules for each position**.

That means:

- More parameters
- More memory
- More training
- Less ability to generalize to sequences of different lengths

With weight sharing:

```
"I"       → W
"love"    → W
"Python"  → W
```

The RNN learns **one general rule** and applies it repeatedly.

Real-life analogy

Imagine a **teacher checking every student's answer using the same marking rule**.

The teacher doesn't create a new marking rule for:

- Student 1
- Student 2
- Student 3

The **same rule** is applied to everyone.

Similarly:

> **RNN = Same weights + different inputs at different time steps**

The Core RNN Equations

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{(t-1)} + b)$$

$$y_t = \text{softmax}(W_{hy} \cdot h_t + c)$$

W_{xh}	Weight used to process the **current input** (e.g., current word).
W_{hh}	Weight used to process the **previous hidden state (memory)**.
W_{hy}	Weight used to convert the **hidden state into the output**.
Shared across every time step	The **same weights are reused** for every word/time step in the sequence.

RNN equation

The hidden state is calculated as:

$$h_t = f(W_x x_t + W_h h_{t-1} + b_h)$$

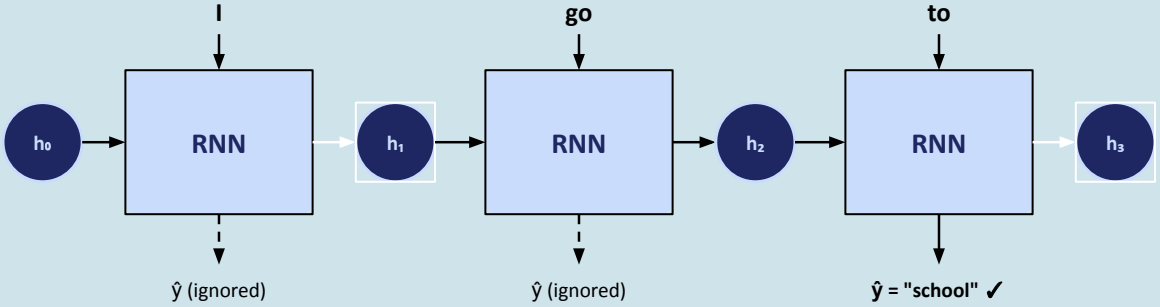
Then output:

$$y_t = g(W_y h_t + b_y)$$

Symbol	Meaning
xt	Input at time step t (e.g. a word / one-hot vector / embedding)
h(t−1), ht	Hidden state from the previous step, and the updated hidden state
Wxh, Whh, Why	Weight matrices — shared across every time step
b, c	Bias vectors for the hidden state and the output
tanh, softmax	Activation functions — tanh squashes the hidden state; softmax turns output scores into probabilities

Predicting the Next Word with an RNN

"I go to ___" → target: "school"



One-Hot Encoding	
Word	**[I, go, to, school]**
I	[1, 0, 0, 0]
go	[0, 1, 0, 0]
to	[0, 0, 1, 0]
school (target)	**[0, 0, 0, 1]**

I → go → to → RNN → "school"

1 Read "I"

RNN reads "I" and stores information in h_1.

2 Read "go"

RNN combines "go" with h_1 to form h_2.

3 Read "to"

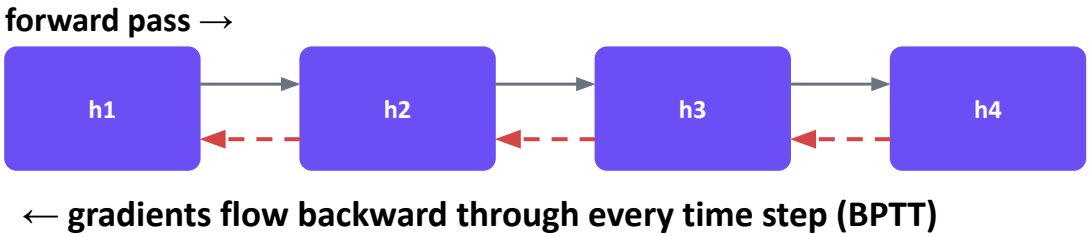
RNN combines "to" with h_2 and predicts "school."

- **d = 3** hidden nodes chosen for this example
- **k = 4** vocabulary size (I, go, to, school)
- **h_0 = [0, 0, 0]** no prior information at the start
- **Target** "school" — the correct next word

Ignored outputs steps 1–2 aren't the goal; only step 3's prediction is used

Training with Backpropagation Through Time (BPTT)

RNNs learn the same way MLPs (Multi-Layer Perceptron) do — backpropagation — but the error at each step depends on every hidden state before it, so gradients must flow backward across time.



BPTT WORKFLOW

- Initialize Wx, Wh, Wy randomly
- Forward propagate to compute predictions
- Compute the loss (cross-entropy, summed over all time steps)
- Backpropagate to compute gradients
- Update weights based on those gradients
- Repeat

Application-Based Question on RNN

Scenario:

A company is developing a chatbot that reads a sentence word by word:

“I really enjoyed this movie.”

The chatbot needs to understand the meaning of the sentence to predict whether the user's review is **Positive or Negative.**

Questions:

1. Why is an RNN suitable for this application?
2. Why does the RNN need to remember previous words?
3. What information is carried by h(t-1)?
4. What is the role of Wx, Wh, and Wy?
5. What could happen if the RNN does not remember previous words?

1. Why is RNN suitable?

An RNN is suitable because it processes words one by one and remembers information from previous words.

For example:

I → really → enjoyed → this → movie

The RNN uses the sequence to understand the complete sentence.

2. Why does RNN need to remember previous words?

Because the meaning of a sentence depends on previous words.

For example:

I → really → enjoyed → this → movie

The word "enjoyed" helps the RNN understand that the review is likely positive.

3. What does h(t-1) represent?

h(t-1) represents the previous hidden state, or simply the memory from the previous step.

Previous information

↓

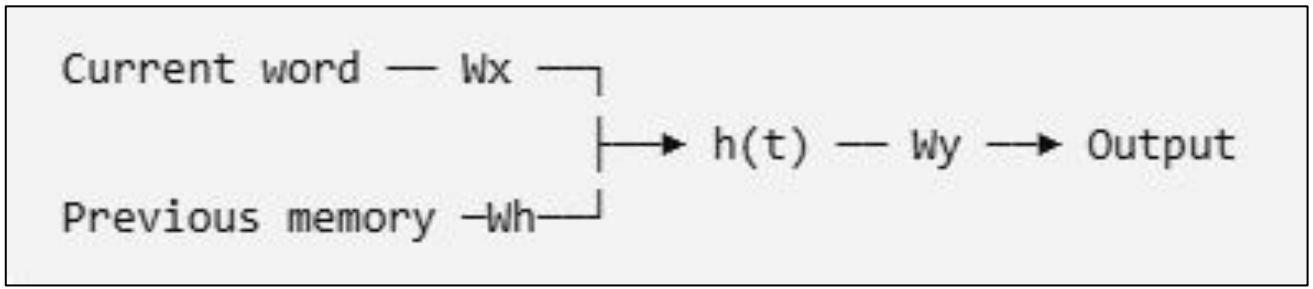
h(t-1)

↓

Current processing

4. What are Wx, Wh, and Wy?

- Wx → weight for the current input
- Wh → weight for the previous hidden state/memory
- Wy → weight used to produce the output



5. What happens if RNN does not remember previous words?

It may not understand the complete context of the sentence.

For example:

"I really enjoyed this movie"

If the RNN only looks at the current word and ignores previous information, it may have difficulty understanding the overall meaning.

Question:

In an RNN, consider the sentence:

“I love Python.”

When the RNN processes the word “Python”, answer:

1. What does Wx represent?
2. What does Wh represent?
3. What does Wy represent?
4. What information is provided by h(t-1)?
5. Draw a simple flow showing how x(t), h(t-1), Wx, Wh, and Wy are connected.

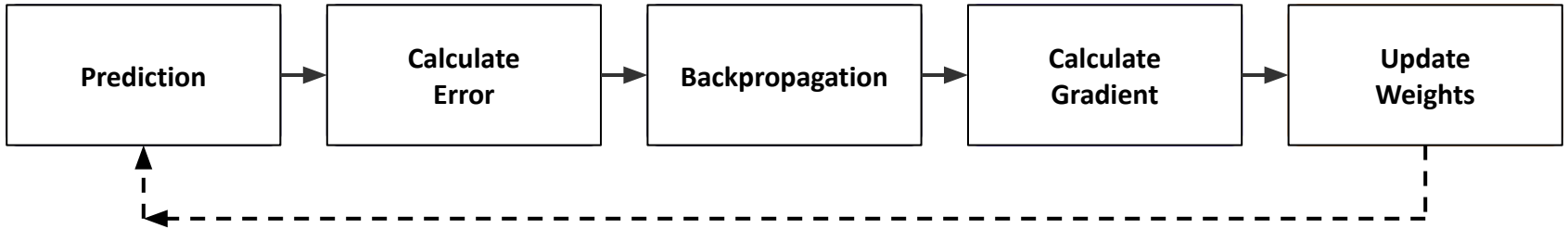
The Vanishing Gradient Problem

Why gradients shrink during backpropagation — and why it makes deep networks and RNNs struggle to learn long-term dependencies.

Understand about the gradient

What Is a Gradient?

A gradient tells a neural network how much, and in which direction, its weights should change to reduce the error. It's the signal that drives learning.



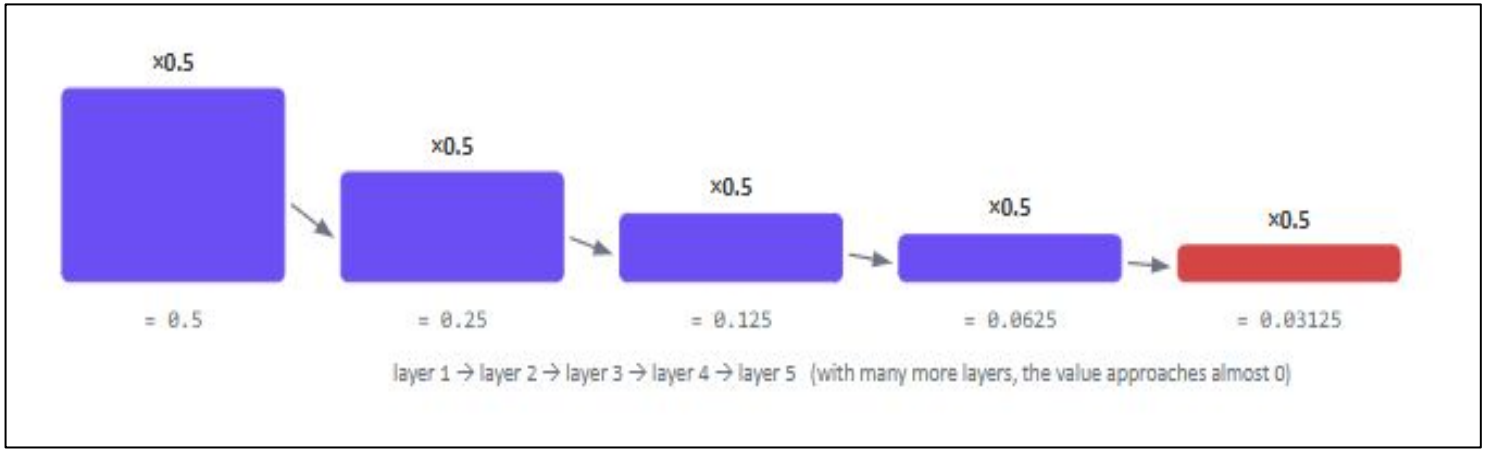
this loop repeats for every training step

The gradient is calculated during backpropagation and used to update every weight in the network — bigger gradients mean bigger, faster corrections.

What Is the Vanishing Gradient?

The vanishing gradient problem occurs when gradients **become extremely small** — close to zero — as they're passed backward through many layers. As a result, weights in the earlier layers learn very slowly, or stop learning altogether.

EXAMPLE — repeated multiplication during backpropagation



Small Gradient → Small Weight Update → Slow Learning → No / Very Little Learning

Easy analogy

Imagine passing a message through many people:

Person 1 → Person 2 → Person 3 → Person 4 → ...

If each person passes only half of the message, eventually almost nothing is left.

That's what happens to the gradient in the vanishing gradient problem.

Why Does It Happen?

Activation Functions

Functions like **Sigmoid** and **Tanh** can give very small gradients.

When these small values are multiplied again and again during backpropagation, the gradient becomes smaller and smaller.

Weight Initialization

The network starts with some initial weights.

If the starting weights are too small, the gradients can also become very small.

Learning Rate

The learning rate controls how much the weights change during learning.

If the learning rate is too small, the network makes very tiny changes and learns very slowly.

Small activation derivatives + too-small weights + too-small learning rate → Very small updates → Slow learning

This is the basic idea behind the vanishing gradient problem

Effect of the Vanishing Gradient

Problem	Effect
Very small gradients	Weights barely change
Earlier layers stop learning	Important features are not learned
Slow training	Model takes longer to train
Poor feature learning	Model may give poor predictions
Difficult deep-network training	Adding more layers may not improve performance

Vanishing Gradient in NLP

This problem is especially important in RNNs, which process a sequence step by step. To predict or understand a later word, the model may need information **from a word much earlier in the sentence.**

"The boy who lives in Mumbai studies NLP."

To understand **"studies"**, the model may need information from an earlier word — **"boy."**



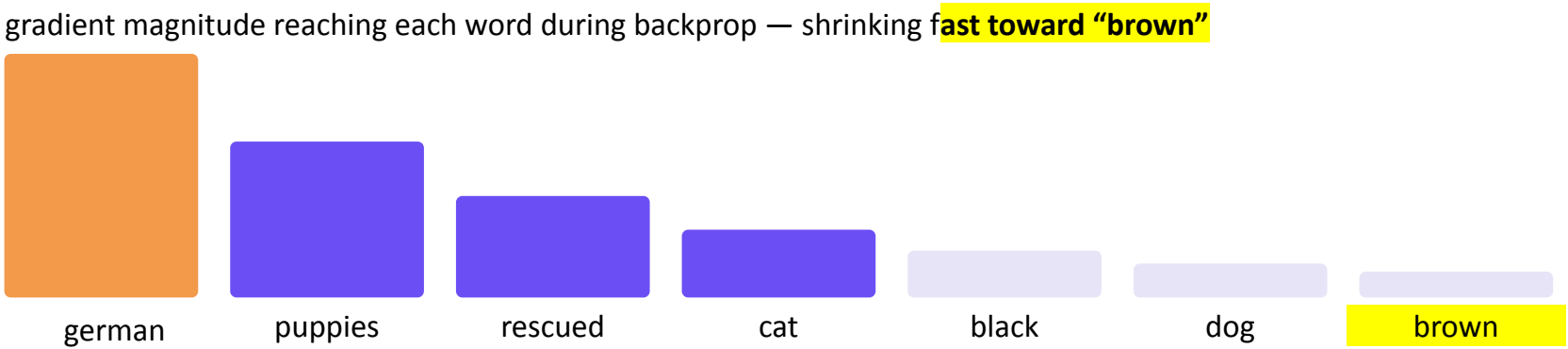
← gradient flows backward through BPTT, shrinking with every step

Earlier words → **Very small gradient** → **Little learning.** The RNN may struggle to remember long-term dependencies like the connection between **"boy" and "studies."**

The Vanishing Gradient Problem

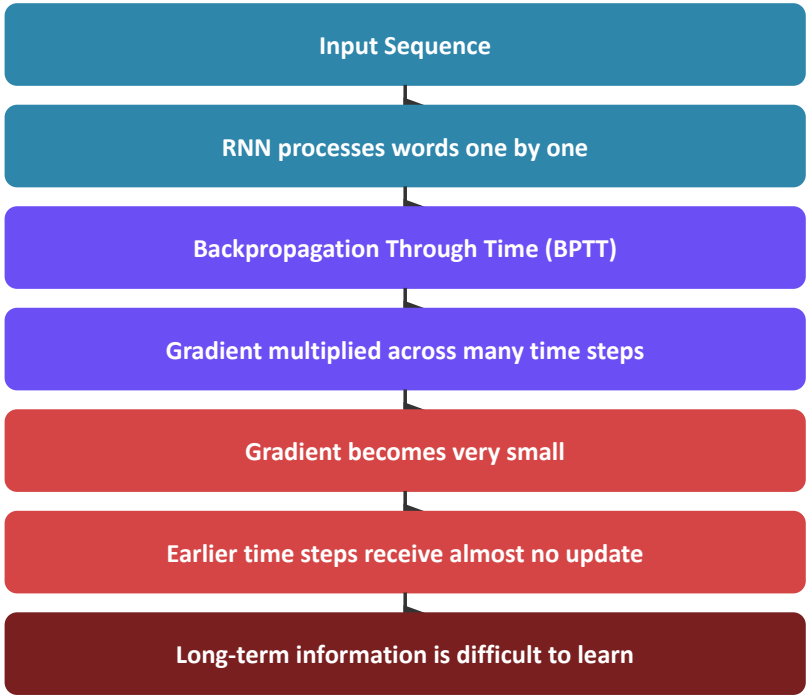
Vanishing gradients make it difficult for an RNN to learn long-term dependencies. Consider a sentence where the words that matter most are far from the word being predicted:

"The big brown dog and the small black cat were both rescued as puppies — he grew up to be a proud german _____"



By the time the gradient signal reaches “brown,” **it has shrunk close to zero** — so the weights barely update to account for it. The model may fail to connect “**brown**” with “**shepherd**” even though a human reader would.

Vanishing Gradient in RNNs — The Flow



Example: Distant Context

> **"The students who studied for many hours yesterday finally _____."**

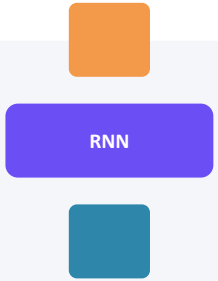
The correct prediction (something like "**passed**" or "**succeeded**") depends on information appearing much earlier in the sentence — "students" and "studied." Because of vanishing gradients, a basic (Vaniilla) RNN may have real difficulty connecting these distant words.

Solution:

- **ReLU:** Reduces saturation and helps gradients flow better.
- **LSTM**: Uses gates and cell state to remember long-term information.
- **GRU:** Simpler gated RNN that retains information with fewer parameters.
- **Weight Initialization:** Xavier/He initialization helps maintain useful gradients.
- **Skip Connections**: Create direct paths for gradients, reducing vanishing-gradient problems.

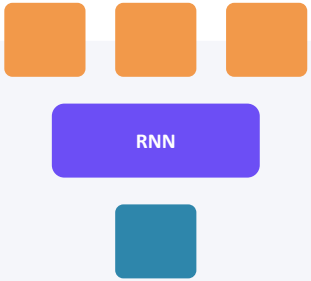
Four Types of RNN Architectures

One-to-One



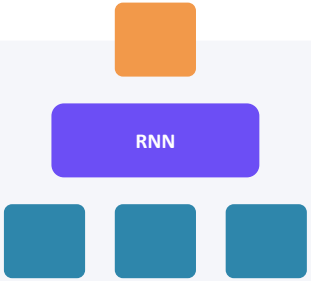
Example: Image classification (Vanilla NN)

One-to-Many



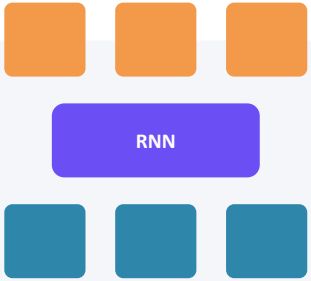
Example: Image captioning (1 image → word sequence)

Many-to-One



Example: Sentiment analysis (word sequence → 1 label)

Many-to-Many



Example: Machine translation (word seq → word seq)

RNN Type	Input → Output	Real-life example	
One-to-One	1 → 1	Face → Person's name	Give one input and get one output.
One-to-Many	1 → Many	Photo → Caption	One photo produces a sequence of words: "A boy is playing cricket."
Many-to-One	Many → 1	Customer reviews → Happy/Unhappy	Many words are read, then one final decision is made.
Many-to-Many	Many → Many	English sentence → Hindi sentence	A sequence of words is converted into another sequence of words.

Think about a teacher and student:

1. One-to-One
Teacher gives one question → Student gives one answer.

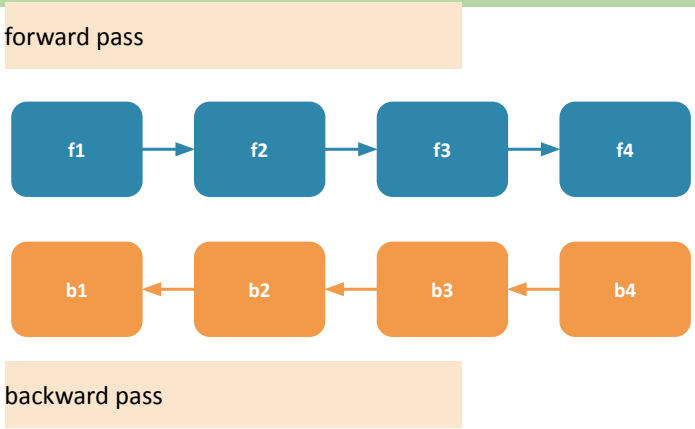
2. One-to-Many
Teacher gives one topic → Student explains it using many sentences.

3. Many-to-One
Student gives many answers → Teacher gives one grade.

4. Many-to-Many
Teacher gives information in many words → Student translates it into many words.

Variants: Bidirectional RNNs, LSTM & GRU

Bidirectional RNN (BiRNN)



Reads the sequence in both directions and combines the two hidden states — useful when the whole sequence is available, e.g. NER, question answering.

LSTM Cell — Three Gates



Forget: discard old info • Input: add new info • Output: decide what to reveal

This selective read-write-forget mechanism lets LSTMs preserve information across many more time steps, directly countering the vanishing gradient problem.

GRU — Gated Recurrent Unit

A simplified LSTM: it merges the forget and input gates into a single **“update gate” a**nd streamlines the output. Fewer parameters, faster to train, and often performs comparably to an LSTM — a common default choice **when compute or data is limited.**

Applications of Recurrent Neural Networks

Language Modeling & Text Generation

Predicting or generating the next word/character in a sequence

Speech Recognition

Converting spoken audio into text by modeling temporal patterns

Machine Translation

Encoding a sentence in one language and decoding it into another

Sentiment Classification

Reading a sequence of words to output an overall sentiment label

Named Entity Recognition

Identifying names of people, organizations, and locations in text

Time-Series Forecasting

Predicting future values from stock prices to weather data

Advantages & Limitations

ADVANTAGES

RNNs remember previous inputs, work well with time-series and sequence data, can handle different sequence lengths, and have a simple structure.

LIMITATIONS

They can suffer from vanishing/exploding gradients,

are slow to train because inputs are processed one by one,

struggle with long sequences, and are often replaced by LSTM, GRU, and Transformer models for complex tasks.

Long Short-Term Memory

RNNs Struggle !!!

In a normal RNN, the model has only one hidden state to remember information. Because of this, it may forget important information when the sequence becomes long.

Vanishing Gradient

As we go backward through many steps, the gradient becomes smaller and smaller.

So, the RNN learns less from older information and may eventually forget it completely.

Exploding Gradient

Sometimes the gradient becomes too large.

This makes the model change its weights too much, causing unstable and difficult training.

LSTM is an improved RNN that can remember important information for a longer time by using a **memory cell and gates.**

Think of the cell state as a highway carrying memory through the sequence.

At every step, three gates act like traffic controllers:

Forget Gate → "What old information should I remove?"
Input Gate → "What new information should I add?"
Output Gate → "What information should I give as output?"

A Solution :LSTM

LSTM helps a neural network remember important information for a long time.

- **Vanishing gradient**: LSTM helps prevent the **learning signal** from becoming too small.
- **Exploding gradient**: LSTM helps control the learning signal from becoming too large.

Learning signal means the information that tells the model how to improve its weights during training.

Learning signal = “What did I learn from my mistake, and how should I change?”

Suppose an RNN is learning to predict the next word:

“I am going to ___”

Model predicts: “School”
Correct answer: “College”

The error/mistake tells the model:

> “Your prediction was wrong. Change your weights.”

This information going backward is called the **learning signal.**

What Is an LSTM?

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network, first proposed by Hochreiter & Schmidhuber (1997), designed specifically to l**earn long-term dependencies in sequential data.**

Cell State C_t

"Long-term memory"

- It carries important information through the whole sequence for a longer time.
- Think of it as a memory highway or conveyor belt. Information can travel through many time steps without changing much.
- The gates decide what information should be added, kept, or removed.

Hidden State h_t

"Short-term memory"

- It stores the important information needed right now and passes it to the next step.
- It is the current output of the LSTM.
- It contains the useful information from the current step and is passed to the next step.

Cell State and Hidden State

Both are needed in LSTM because they do different jobs.

Example

Sentence:

“I went to school because I had an exam.”

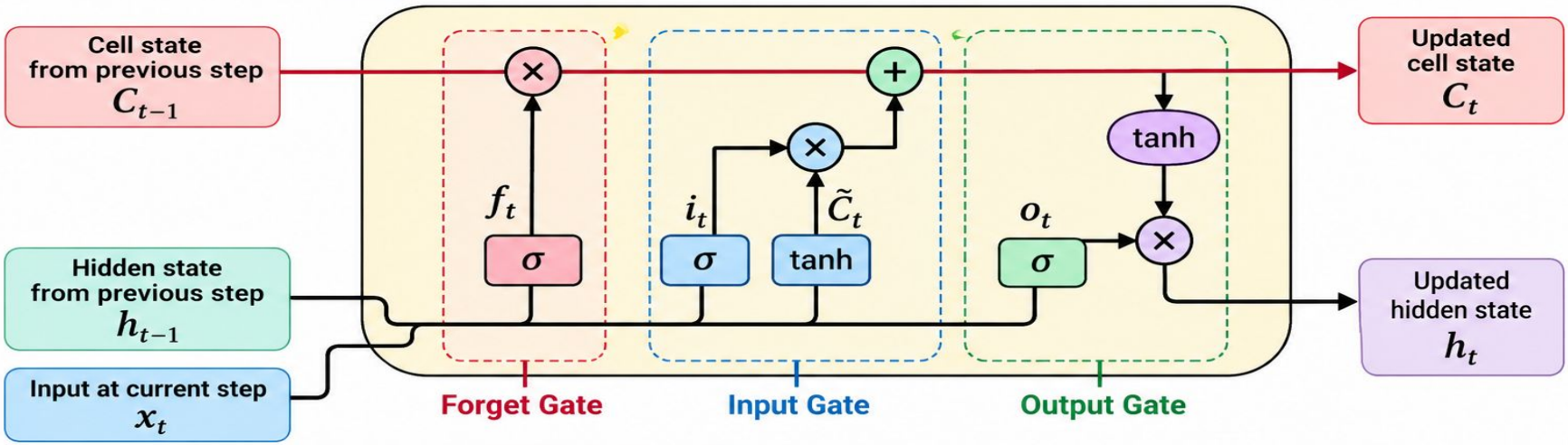
At the word “exam”:

- **Cell State** → keeps the important long-term information: “exam”.
- **Hidden State** → carries the information needed right now to predict what comes next.

So:

Cell State = Long-term memory
Hidden State = Current output

LSTM (Long Short-Term Memory)



Forget Gate

Decides what old information to remove from the cell state.

$$f_t = \sigma(W_f\,[h_{t-1}, x_t] + b_f)$$

If f_t is 0 → forget
If f_t is 1 → keep

Input Gate

Decides what new information to add to the cell state.

$$i_t = \sigma(W_i\,[h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C\,[h_{t-1}, x_t] + b_C)$$

i_t controls how much new information to add.

Output Gate

Decides what information to give as output (hidden state).

$$o_t = \sigma(W_o\,[h_{t-1}, x_t] + b_o)$$

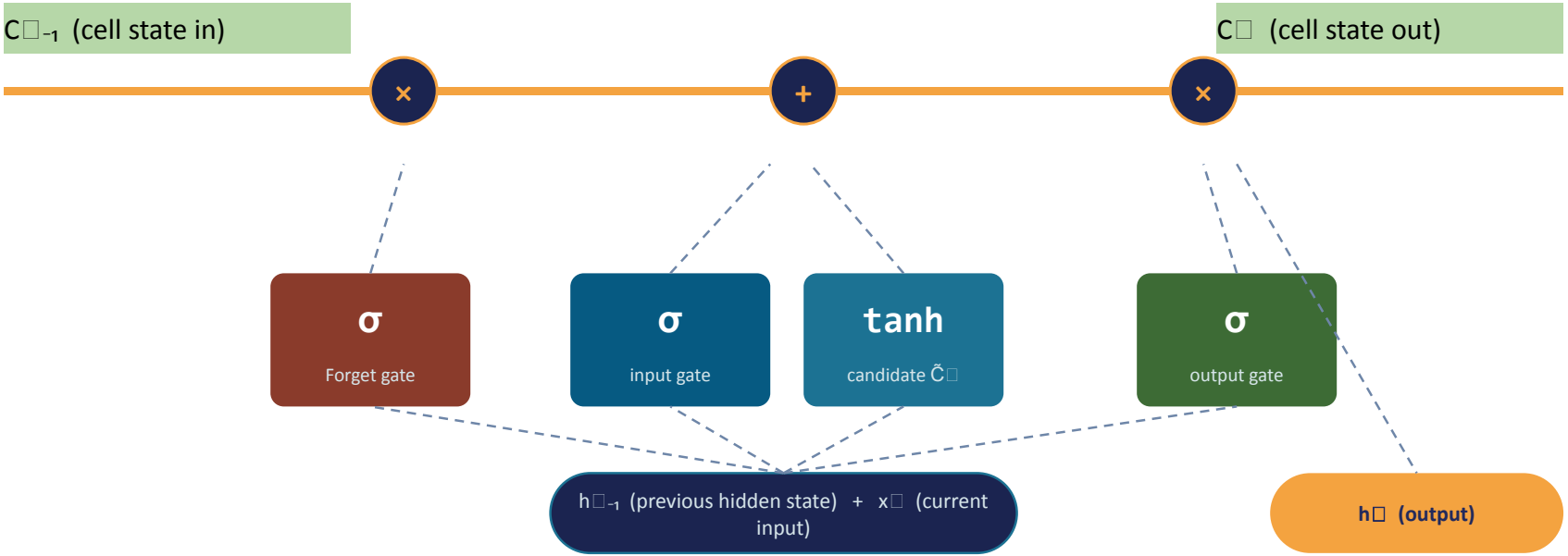
o_t controls how much information to output.

Cell State – Memory Highway

Carries important information through many time steps.

Helps LSTM remember important information for a longer time.

One time step: three gates control a protected cell-state highway



Think of the cell state as a memory highway running through the LSTM.

- Cell state moves information forward from one time step to the next.
- The Forget Gate decides what old information should be removed (multiplied).
- The Input Gate decides what new information should be added.
- The cell state is not squeezed or compressed by functions **like sigmoid or tanh at every step**

The cell state is like a memory highway where information can travel a long distance without **getting weakened too much.** The gates act like traffic **controllers, deciding what to remove and what to add.**

Understanding of the sigmoid and tanh functions in gate

Sigmoid (σ) → used in the Forget, Input, and Output gates.

It gives a value between 0 and 1, like a switch:

- 0 → forget/block
- 1 → keep/allow

Tanh → used to create and control the new information stored in the cell.

It gives values between -1 and +1, so the information can have both positive and negative values.

Sigmoid → decides "HOW MUCH should pass?"

It works like a gate/switch: 0 = block, 1 = allow.

Tanh → decides "WHAT information should be stored?"

It creates values between −1 and +1.

Suppose new information is **"I am going to college."**

Sigmoid: "Should I let this information enter memory?" → Yes, 0.9

Tanh: "What should the information look like?" → gives a value between −1 and +1.

So:

Sigmoid controls the door ; tanh prepares the information .

Operations performed on gate

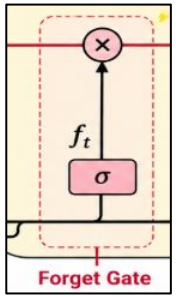
1. × = Multiplication

The × perform element-wise multiplication.

For example:

Cell state × Forget Gate

$$C_{t-1} \times f_t$$



If:

- Cell state = 10
- Forget gate = 0.8

Then:

10 × 0.8 = 8

So, the forget gate allows 80% of the old information to continue.

2. + = Addition

The + combines the old information and new information.

$$C_t = (C_{t-1} \times f_t) + (i_t \times \tilde{C}_t)$$

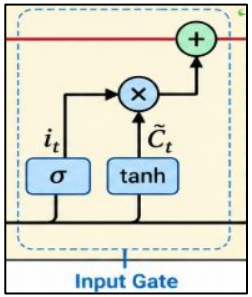
For example:

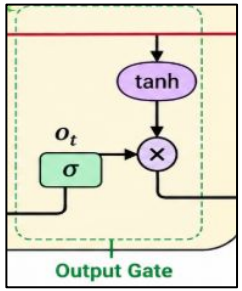
- Old information = 8
- New information = 3

Then:

8 + 3 = 11

This creates the updated cell state CtC_t.





3. × = Multiplication

1. Cell information ⟶ tanh(C□)
2. Output Gate value ⟶ o□ from sigmoid

The operation is:

$$h_t = o_t \times \tanh(C_t)$$

Example

Suppose:

- tanh(C□) = 0.8 ⟶ available information
- o□ = 0.5 ⟶ output gate allows 50%

Then:

0.8 × 0.5 = 0.4

So 0.4 is passed to the hidden state h_t.

The multiplication controls how much of the cell's current memory is exposed as output.

Gate 1 — The Forget Gate

Deciding what to erase from long-term memory

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

The forget gate looks at the previous hidden state h_{t-1} and the current input x_t, and passes them through a sigmoid. The output is a vector of values between 0 and 1 — one per cell-state dimension.

≈ 0	“Forget this.”
≈ 1	“Keep this.”

EXAMPLE

"Bob is a nice person. Dan, on the other hand, is evil."

While reading the first sentence, the LSTM remembers information about Bob. When it reaches the full stop (.), the forget gate decides that Bob's information is no longer needed.

It removes that old information and makes space to remember information about Dan.

Gate 2 — The Input Gate

Deciding what new information is worth remembering

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

Two things happen in parallel: i_t (sigmoid) decides how important the new information is, while $\tilde{C}_t$ (tanh) proposes what that new information actually looks like, as values between −1 and +1.

EXAMPLE

"Bob knows swimming. He told me over the phone that he had served in the navy for four years."

The LSTM reads information about Bob.

- "Served in the navy for four years" → important information, so the Input Gate allows more of it into memory.
- "Told me over the phone" → less important information, so the Input Gate allows less of it.

Input Gate = decides what new information is important enough to remember.

Updating the Cell State

Combining what we kept with what we're adding

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$f_t \odot C_{t-1}$

The old cell state, filtered by the forget gate. Irrelevant history is scaled toward zero; important history passes through.

$i_t \odot \tilde{C}_t$

The new candidate information, scaled by how important the input gate judged it to be.

- The new cell state is updated using simple multiplication and addition.
- This helps the information and learning signal flow through many time steps more easily.
- In a normal RNN, the signal can become smaller and smaller, causing the model to forget old information.
- In LSTM, the forget gate controls how much information continues forward. Therefore, important information can stay in memory for a longer time.

Gate 3 — The Output Gate

Deciding what to reveal as this step's output

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

The updated cell state is squashed through **tanh** (into the range −1 to +1), then filtered by the output gate o_t. The result, h_t, becomes both this step's output and the short-term memory carried into the next step.

LSTM : Working with an Example

Example : Rahul went to the market because he wanted to buy some fruits. He returned home.

1 Input : "Rahul"

The LSTM receives the word "Rahul". The **Input Gate** stores this important information in the cell state.

Cell State
Rahul

→

2 "went to the market"

The **Forget Gate** checks the old information. Since Rahul is still useful, it keeps this information in the cell state.

Cell State
Rahul + market

→

3 "because he wanted to buy some fruits"

The **Input Gate** decides what new information to add. "Fruits" is important, so it is stored in the cell state along with the previous information.

Cell State
Rahul + market + fruits

→

4 Read "He"

Now the model sees the word "He". It needs to know who "he" refers to.

The **Output Gate** looks at the cell state and selects the useful information.

→

5 "He returned home."

The **Output Gate** brings out the information about Rahul from the cell state. So the model understands "He" = Rahul and continues the sentence correctly.

Cell State
Rahul + market + fruits

Role of Each Gate in This Example

Gate	What it does	In this example
Forget Gate	Removes unnecessary information from the cell state.	Keeps information about Rahul because it is still useful.
Input Gate	Adds important new information to the cell state.	Stores information about "market" and "fruits".
Output Gate	Selects information needed now from the cell state.	Uses Rahul's information to understand "He" and predict the next words.

Key Takeaway

LSTM remembers important information, forgets unnecessary information, and brings out the right information when needed.

LSTM vs. RNN vs. GRU



Feature	Simple RNN	LSTM	GRU
Memory	Single hidden state	Cell state + hidden state	Single hidden state
Gates	None	3 (forget, input, output)	2 (update, reset)
Long-range dependencies	Poor (vanishing gradient)	Strong	Good, slightly less precise
Parameters / speed	Fewest, fastest	Most, slowest to train	Fewer than LSTM, faster
Best fit	Very short sequences	Large datasets, long sequences	Smaller datasets, real-time use

Gated Recurrent Unit (GRU) Networks

GRU, explained over a cup of tea

How a Gated Recurrent Unit decides what to remember, what to forget, and what to learn

The analogy

Think of a GRU as a person making tea every morning. They remember past cups, then decide what to keep, forget or update.

YESTERDAY

Made tea with less sugar

→

TODAY

Making tea again: what should I remember?

STEP 1 · PREVIOUS KNOWLEDGE

The hidden state h_{t-1}

WHAT YOU REMEMBER

"Yesterday, I used less sugar."[1]

The hidden state is the memory carried over from the previous step: everything the GRU remembers so far.

STEP 2 · WHAT SHOULD I KEEP?

The update gate



SITUATION: YOU ENJOYED YESTERDAY'S TEA

“I liked yesterday's tea, so I will keep the less-sugar preference.”

Update gate

The update gate decides how much old information to retain and how much new information to add.

STEP 3 · WHAT SHOULD I FORGET?

The reset gate



SITUATION: THIS TEA IS FOR SOMEONE WHO LIKES MORE SUGAR

“Forget my usual less-sugar preference because this tea is for someone else.”

The reset gate lowers the importance of old memory when it isn't relevant to the new situation.

STEP 4 · THE RESULT

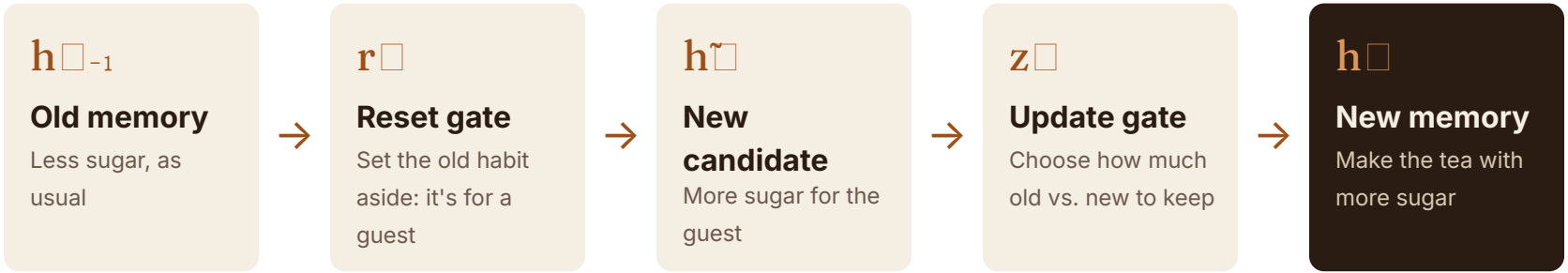
The new hidden state h_t

USEFUL OLD INFO + THE NEW SITUATION

“Today, make the tea with more sugar.”

The GRU combines what is still useful from memory with the new situation to form its updated memory.

The Flow

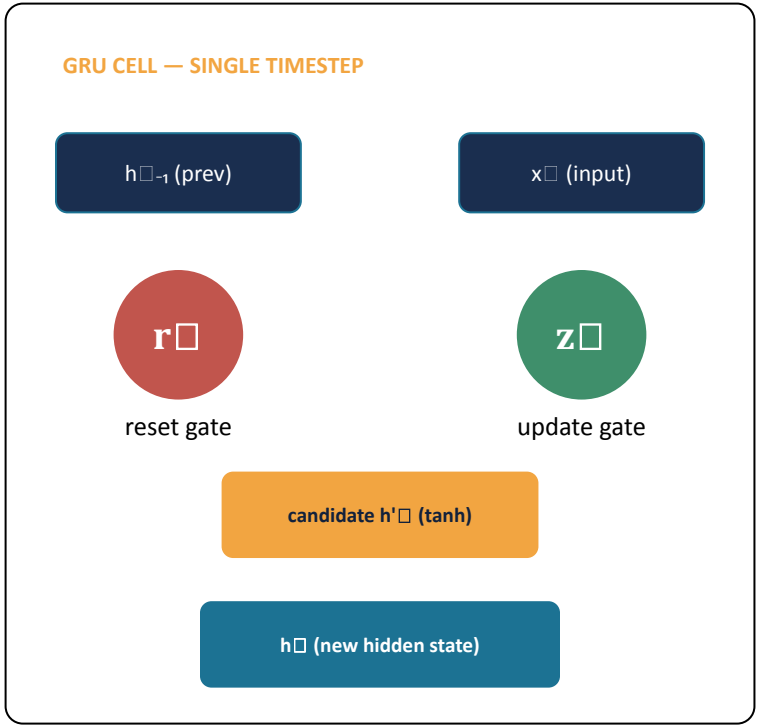


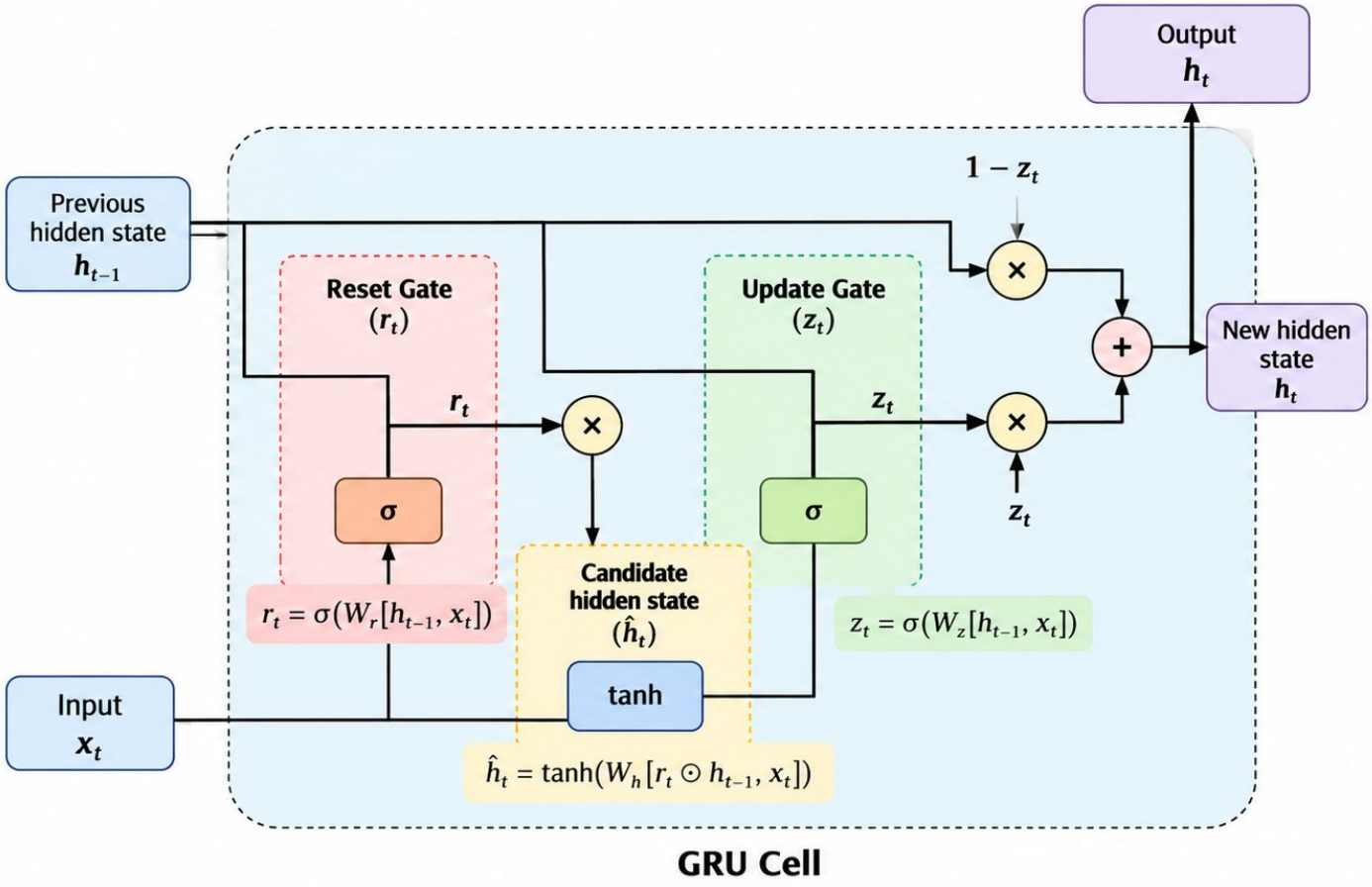
The reset gate shapes the new candidate; the update gate blends that candidate with the old memory.

What Is a Gated Recurrent Unit?

A GRU is a recurrent neural network unit that regulates information flow using two learned gates instead of a separate memory cell.

- Introduced by Cho et al. in 2014 for statistical machine translation
- Only one state vector: the hidden state h□ (no separate cell state C□)
- **Update gate z□** — how much of the past to keep vs. replace
- **Reset gate r□** — how much of the past to forget when forming new candidate info
- Fewer parameters than LSTM → faster training, less overfitting risk on small data





GRU Equations

Reset gate:

$$r_t = \sigma(W_r[h_{t-1}, x_t])$$

Update gate:

$$z_t = \sigma(W_z[h_{t-1}, x_t])$$

Candidate hidden state:

$$\hat{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t])$$

Final hidden state:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \hat{h}_t$$

Symbols

x_t	:	input at time t
h_{t-1}	:	previous hidden state
h_t	:	new hidden state (also output)
r_t	:	reset gate (0 to 1)
z_t	:	update gate (0 to 1)
$\hat{h}_t$	:	candidate hidden state
σ	:	sigmoid activation
$tanh$	:	tanh activation
$\odot$	:	element-wise multiplication
$[h_{t-1}, x_t]$	:	concatenation of h_{t-1} and x_t
W_r, W_z, W_h	:	learnable weight matrices

Example:

“Rahul went to the market because he wanted to buy fruits.”

Suppose the GRU is reading the sentence word by word. When it reaches “**he**”, it needs to remember that “**he” refers to Rahul.**

The GRU uses the previous hidden state and the current word to decide what information to retain, forget, and add.

1. Input to the GRU

At a particular time step:

- Current input = **"he"** → x_t
- Previous hidden state = information learned from **"Rahul went to the market because"** → h_{t-1}

So the GRU receives:

$$x_t = \text{“he”}$$

and

$$h_{t-1} = \text{previous context}$$

Think of h_{t-1} as the GRU's **memory of what has happened so far**.

2. Both x_t and h_{t-1} go to the gates

The diagram shows that the current input and previous hidden state are used to calculate **two gates**:

Reset Gate

$$r_t = \sigma(W_r[h_{t-1}, x_t])$$

Update Gate

$$z_t = \sigma(W_z[h_{t-1}, x_t])$$

The sigmoid function produces values between **0 and 1**.

For example, imagine the GRU calculates:

$$r_t = 0.8$$

and

$$z_t = 0.7$$

These numbers are only for understanding the process.

3. Reset Gate — "How much old information should I use?"

The **reset gate** r_t controls how much of the previous hidden state should be used when creating the new candidate information.

Suppose:

$$r_t = 0.8$$

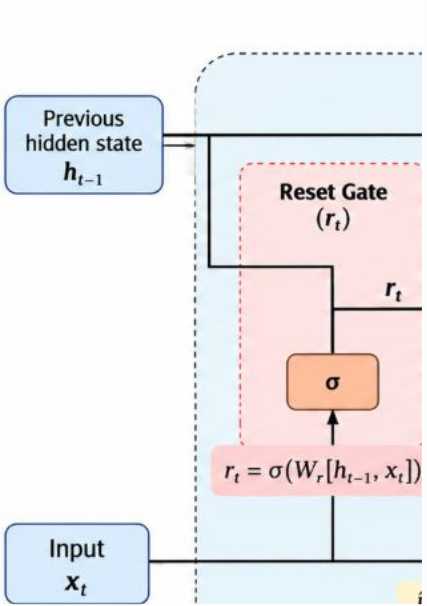
This means:

> "Use a large portion of the previous information."

The GRU performs:

$$r_t \odot h_{t-1}$$

where $\odot$ means **element-wise multiplication**.



Simple analogy

Previous memory:

> “Rahul went to the market.”

Current word:

> “he”

The reset gate says:

> “The previous information about Rahul is useful, so don't ignore it.”

Therefore, information about Rahul can contribute strongly to the new candidate state.

4. Candidate Hidden State

Now the GRU creates a **candidate hidden state**:

$$\hat{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t])$$

This is essentially the GRU asking:

> **"Based on my previous memory and the current word, what new information should I create?"**

For our example:

Previous context:

> Rahul went to the market...

Current word:

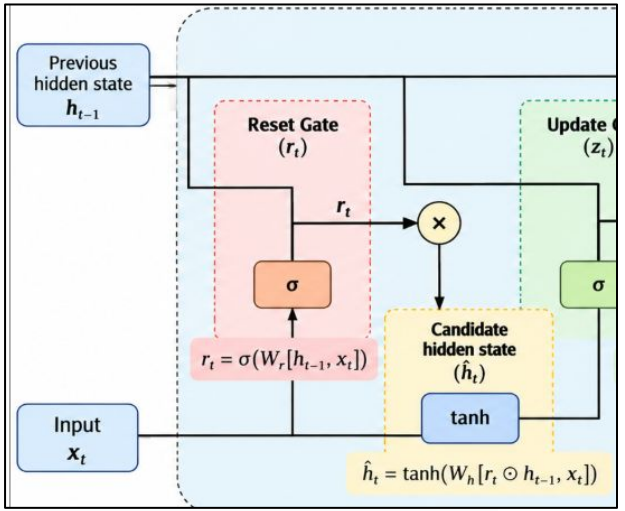
> he

The candidate state may contain information such as:

> "he is probably referring to Rahul."

So:

$$\hat{h}_t = \text{new candidate information}$$



5. Update Gate — The Most Important Step

Now look at the **update gate** z_t.

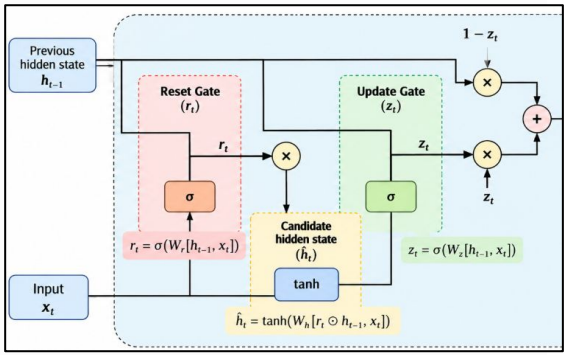
Suppose:

$$z_t = 0.7$$

The update gate determines how much of the **candidate information** should enter the new hidden state.

The final equation is:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \hat{h}_t$$



For $z_t = 0.7$:

$$h_t = 0.3h_{t-1} + 0.7\hat{h}_t$$

So the GRU is effectively saying:

> **30% old information + 70% new candidate information**

6. Why do we have $1 - z_t$?

This is important when explaining the diagram.

The update gate has two paths:

Old information

$$(1 - z_t) \odot h_{t-1}$$

New information

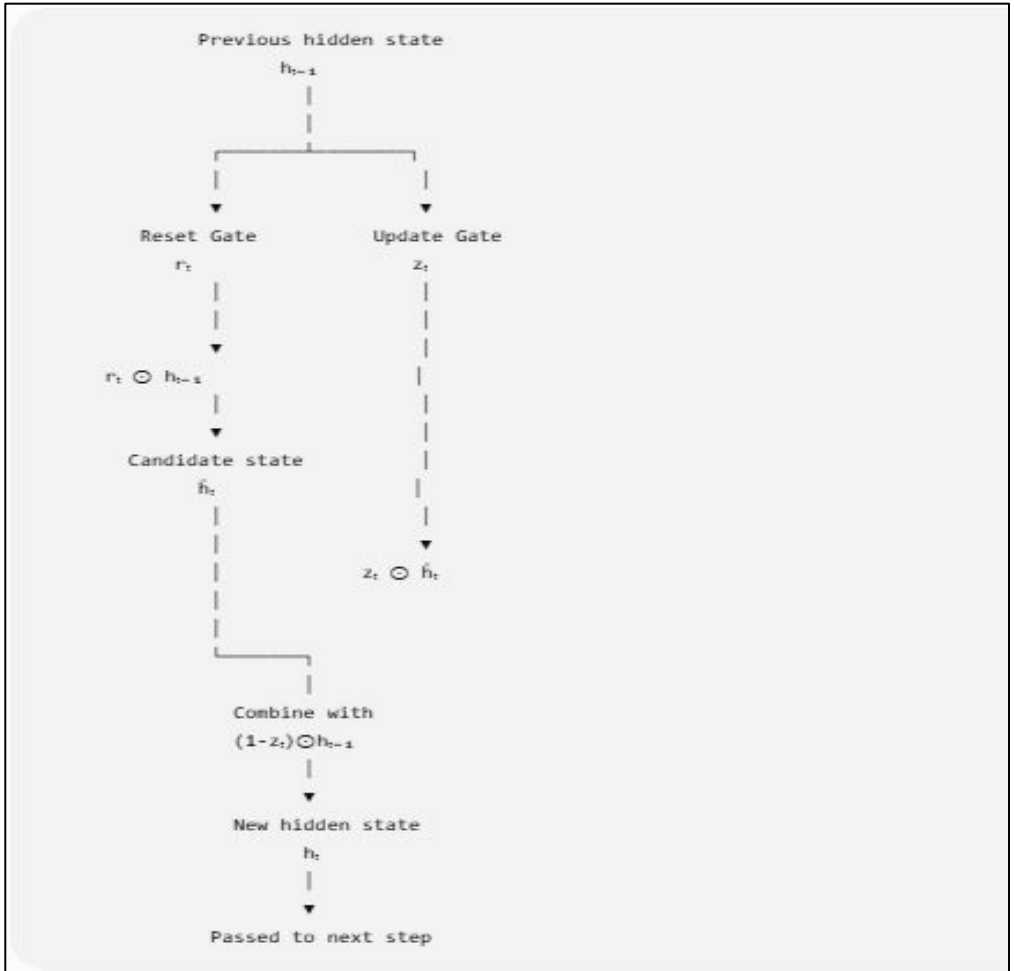
$$z_t \odot \hat{h}_t$$

Then they are added:

$$\boxed{h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \hat{h}_t}$$

Therefore, the GRU decides how much **old memory** and **new information** should make up the new hidden state.

7. Complete Flow of the Diagram



And the current input x_t is used by **both gates** and the **candidate calculation**.

GRU vs. LSTM

Aspect	LSTM	GRU
Gates	3 — Input, Forget, Output	2 — Update, Reset
Memory unit	Separate cell state C□ + hidden state h□	Hidden state h□ only
Parameters	More (≈4× hidden×input+hidden weights)	Fewer (≈3× hidden×input+hidden weights)
Training speed	Slower — more computation per step	Faster — leaner architecture
Data efficiency	Needs more data to avoid underfitting gates	Often performs well with less data
Long-range memory	Very strong — dedicated cell highway	Strong — comparable on many tasks
Best for	Very long sequences, complex dependencies	Faster iteration, smaller/medium datasets

Advantages & Disadvantages

Advantages	Disadvantages
Faster training	Simpler than LSTM
Good at remembering information	May struggle with very complex sequences
Fewer parameters	Can overfit small datasets
Easy to implement	Processes data sequentially

THANKS